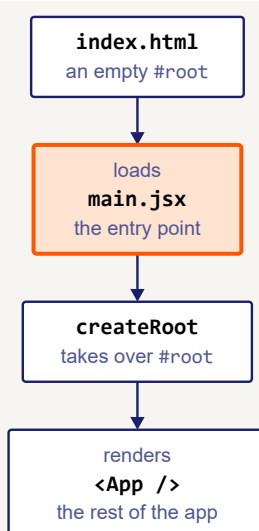


1. **Components and JSX:** A Vite project: from npm create to the browser · A component: a function from data to UI · Describe the result, not the steps · JSX is JavaScript: what it compiles to · Elements, rendering and the DOM · Keep rendering pure: compute, do not change · JSX rules: one parent, closed tags, JavaScript names · Expressions in braces: what can go in {...}
2. **Props and composition:** Props: passing data into a component · Props are read-only: data flows down · children: components that wrap other content · Composing components and splitting files
3. **Lists and conditions:** Rendering lists with map · Keys: telling list items apart · Conditional rendering: if, ternary and &&
4. **Events and forms:** Events: pass a function, do not call it · Controlled inputs: value and onChange · Uncontrolled inputs: defaultValue and FormData · Forms: onSubmit and preventDefault · Forms with several fields and field errors

Components and JSX

A Vite project: from npm create to the browser

Vite is the build tool that turns a React project into something a browser can run: `npm run dev` serves it while you work, reloading on every save, and `npm run build` produces the files to publish.



The page is empty until main.jsx hands #root to React.

- npm install** Downloads React and Vite into node_modules/ (see [package.json](#)).
- npm run build** Compiles and bundles everything into dist/: plain HTML, CSS and JavaScript that any static server can host. `npm run preview` serves that build locally.

```

# Create, install and run
npm create vite@latest my-app -- --template react
cd my-app
npm install
# http://localhost:5173, reloads on save
npm run dev

<!-- index.html: the only HTML page -->
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>

// src/main.jsx: connect React to #root
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './index.css';
  
```

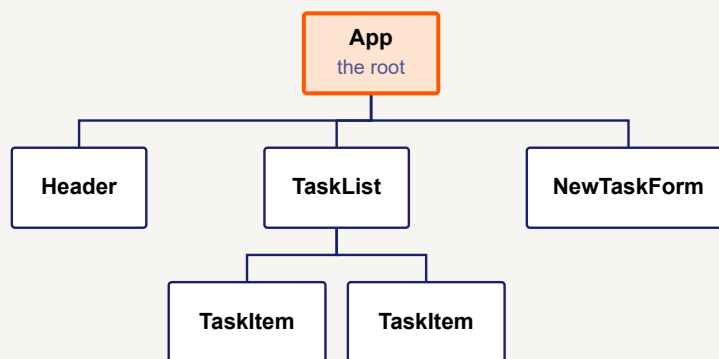
```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Example. Open the browser's developer tools on a running Vite app and look at the Elements panel: `index.html` contains only `<div id="root">`, and everything inside it was created by React. View the page source instead (Ctrl+U) and `#root` is empty: the content does not exist until the JavaScript runs.

Common mistake. Opening `index.html` directly from the file explorer (a `file://` address) and getting a blank page. The browser cannot compile JSX or resolve the imports by itself; the page only works through `npm run dev` (development) or after `npm run build` from a server. A blank page with Vite running usually means an error: open the console (F12).

A component: a function from data to UI

A **component** is a function that receives data and returns a description of a piece of the interface; an app is a tree of components, each responsible for one part of the screen.



A screen is a tree of components; `TaskItem` is written once and used for every task.

- **A function:** data in, a description of UI out. In React it returns **JSX** (see [JSX is JavaScript](#)).
- **A capital letter:** `TaskItem`, used as a tag, `<TaskItem />`. React calls the function when it needs that part of the page; you never call it yourself.
- **A tree:** one root component, usually `App`, renders the others. Each piece is small enough to read in one go, reusable (the same `TaskItem` for every task) and changeable without touching the rest.

```
function TaskItem() {
  return <li>Buy milk</li>;
}

function App() {
  return (
    <main>
      <h1>My tasks</h1>
      <ul>
        <TaskItem />
        <TaskItem />
      </ul>
    </main>
  );
}
```

Example. In the Try it box, App uses `TaskItem` twice, so React calls the `TaskItem` function twice and the list shows two items. Rename `TaskItem` to `taskItem` (both the function and the tags): the items disappear, and React warns in the console about the casing of the tag. A lowercase tag means "an HTML element called `taskitem`", not "my component". Both items say "Buy milk" because the component has no input yet: [Props](#) fixes that.

Common mistake. Calling a component as a function, `{TaskItem()}`, instead of rendering it as `<TaskItem />`. It may seem to work, but React no longer sees a separate component, so anything React keeps for each component (its state, see [useState](#)) ends up attached to the parent and breaks in confusing ways. Render components with the tag syntax and let React call them.

Describe the result, not the steps

With the DOM API you write the **steps** that change the page; with components you write **what the page should look like** for the current data, and the library works out the steps. This is **declarative UI**.

Two ways to keep a task list on screen

	Imperative (the DOM API)	Declarative (a component)
You write	The steps: create, fill, append, update the counter	What the page looks like for these tasks
A task is added	You run every step again, by hand	You add it to the data; React works out the steps
A step is forgotten	The page lies about the data: a stale counter, a "no tasks" message that stays	Cannot happen: there is one description
Changes go through	The DOM	The data (state)

```
// Imperative (DOM API): every change is a list of steps
const ul = document.querySelector('#tasks');
const li = document.createElement('li');
li.textContent = 'Call Ana';
ul.appendChild(li);
document.querySelector('#count').textContent = ul.children.length;

// Declarative (a React component): the page for the current data
function TaskList({ tasks }) {
  return (
    <>
      <p>{tasks.length} tasks</p>
      <ul>
        {tasks.map((t) => <li key={t.id}>{t.title}</li>)}
      </ul>
    </>
  );
}
```

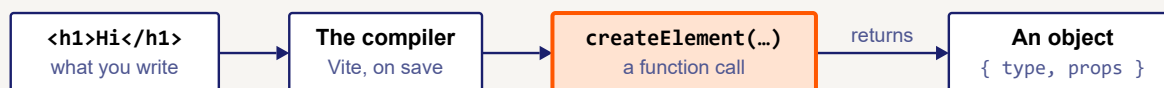
Example. Add a task in each version of the code. Imperative: you append an `<li>` **and** remember to update the counter (and the empty message, and the "all done" badge...). Declarative: you add the task to the array; React calls `TaskList` again with the new array, sees one more `<li>` and a new number in the `<p>`, and changes exactly those two things in the DOM. (The key on each `<li>` is explained in [Keys](#).)

Common mistake. Mixing both styles: calling `document.querySelector(...).textContent = ...` inside a React app to "quickly" update something. React does not know about that change; the next time it re-renders the component it puts back what the component describes, and your edit vanishes (or React gets confused about the DOM it manages). Change the data and let the component describe the result.

JSX is JavaScript: what it compiles to

JSX is an HTML-like syntax inside JavaScript: a compiler turns every tag into a function call,

`React.createElement(type, props, ...children)`, which returns a plain object describing that piece of UI.



A tag is a function call that returns an object.

type A string for an HTML tag ('h1'), or the component function itself (TaskItem, no quotes). That is why components start with a capital letter: the compiler reads a capitalised tag as a variable.

props An object with the attributes; null when there are none.

children Everything between the opening and closing tags, compiled the same way, so nested tags become nested calls.

```

// What you write
const el = (
  <ul className="tasks">
    <li>Buy milk</li>
    <TaskItem title="Call Ana" />
  </ul>
);

// What the compiler writes (classic form)
const el = React.createElement('ul', { className: 'tasks' },
  React.createElement('li', null, 'Buy milk'),
  React.createElement(TaskItem, { title: 'Call Ana' })
);
  
```

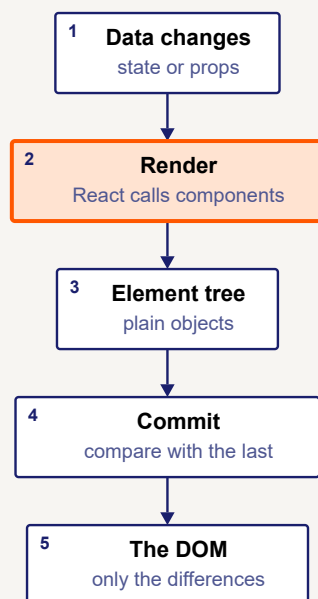
Example. In the JSX viewer, pick the example "A component with props and children". Pane 1 shows `React.createElement(Card, { title: "Today" }, React.createElement('p', null, "Buy milk"))`: the type of the outer call is the **function** `Card`, not a string, and the `<p>` is compiled first and passed as an argument. Pane 2 shows the object that call returns.

Common mistake. Thinking JSX is HTML and copying HTML habits into it: `class=`, `for=`, unclosed `<input>`, `onclick="save()"` as a string. JSX is JavaScript, so it follows JavaScript names and syntax: `className`, `htmlFor`, `<input />`, `onClick={save}`. See [JSX rules](#).

Elements, rendering and the DOM

A **React element** is the plain object a JSX tag produces, `{ type, key, props }`; **rendering** is React calling

your components to get a tree of elements, then making the DOM match it.



Your component runs in the render phase; only the commit touches the DOM.

```

function Greeting({ name }) {
  console.log('Greeting renders for', name);
  return <p>Hello, {name}!</p>;
}

function App() {
  const el = <Greeting name="Ana" />;
  console.log('The element is an object:', el.type.name, el.props);
  return (
    <div>
      {el}
      {el}
    </div>
  );
}
  
```

Example. In the Try it box, look at the console: App creates the element once, and it is only an object with type (the Greeting function) and props ({ name: 'Ana' }). Greeting runs **twice**, once for each place the element appears in the tree, because rendering means React calling the component for every place it is used.

Common mistake. Expecting a component to run once, like a script at the end of <body>. It runs on every render, possibly many times; a console.log in it prints again and again (and twice per render in development, see [StrictMode](#)). Code that must run once, or that talks to the outside world (fetching, timers), does not belong in the body of the component.

Keep rendering pure: compute, do not change

A component's body should only **compute** its JSX from its inputs (props and state): no changing variables outside it, no editing props or arrays in place, no requests or timers.

In the body of a component

Fine	Not in the body
Compute with filter, map, toSorted, spread copies	Mutate with push, sort, props.x = ...
Read props and state	Change a variable declared outside the component
Return JSX	Fetch data, start a timer, write to localStorage

```

const tasks = [
  { id: 't1', title: 'Water the plants', done: false },
  ...
]
  
```

```

    { id: 't2', title: 'Buy milk', done: true },
    { id: 't3', title: 'Call Ana', done: false },
  ];

function TaskSummary({ tasks, search }) {
  // Derived values: computed every render, never stored.
  const pending = tasks.filter((t) => !t.done).length;
  const shown = tasks
    .filter((t) => t.title.toLowerCase().includes(search.toLowerCase()))
    .toSorted((a, b) => a.title.localeCompare(b.title));
  return (
    <>
    <p>{pending} of {tasks.length} left</p>
    <ul>{shown.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </>
  );
}

function App() {
  return <TaskSummary tasks={tasks} search="a" />;
}

```

Example. The box sorts with `toSorted`, which returns a new array. Replace it with `.sort(...)` after the filter: still fine, because filter already made a copy. But `tasks.sort(...)` directly would reorder the shared `tasks` array, so any other component reading it would also see the new order, a change nobody asked for.

Common mistake. Storing a derived value in its own state and trying to keep it updated ("when the tasks change, also update `pendingCount`"). Two sources of truth eventually disagree. If it can be computed from what you already have, compute it during render.

JSX rules: one parent, closed tags, JavaScript names

JSX must be valid JavaScript: a component returns **one** root (a fragment `<>...</>` groups without adding an element), **every tag is closed**, and attributes use JavaScript names.

HTML habits and their JSX form

HTML	JSX	Why
<code>class="x", for="email"</code>	<code>className="x", htmlFor="email"</code>	<code>class</code> and <code>for</code> are reserved words in JavaScript
<code><input>,
</code>	<code><input />,
</code>	The compiler needs to know where children end; any element may self-close
<code>onclick="save()", tabindex</code>	<code>onClick={save}, tabIndex</code>	Multi-word names are camelCase; <code>aria-*</code> and <code>data-*</code> keep their dashes
<code>style="color: red"</code>	<code>style={{ color: 'red', marginTop: 8 }}</code>	An object: outer braces open JavaScript, inner ones are the object; numbers get <code>px</code>
<code><!-- note --></code>	<code>{/* note */}</code>	A JavaScript comment inside braces

```

function App() {
  const late = true;
  return (
    <>
    /* a fragment: no extra element in the DOM */
    <label htmlFor="email" className="label">Email</label>
    <input id="email" type="email" />
    
    <p style={{ color: late ? 'crimson' : 'green', marginTop: 8 }}>
      {late ? 'Late' : 'On time'}
    </p>
    </>
  );
}

```

Example. In the Try it box, delete the / of `<input ... />`: the preview stops updating and a syntax error points at the line, because the compiler cannot tell where the input ends. Change `className` to `class`: it still renders, but React warns Invalid DOM property 'class'. Did you mean 'className'? in the console.

Common mistake. Wrapping everything in an extra `<div>` "to make the error go away". It works, but it adds a real element to the page: inside a `<ul>` you end up with `<div>` between `<ul>` and `<li>` (invalid HTML), and in a flex or grid container the wrapper becomes the only item, breaking the layout. Use a fragment when you only need grouping.

Expressions in braces: what can go in {...}

Braces open a window to JavaScript inside JSX: they take any **expression** (something that produces a value), but not statements such as `if` or `for`.

What renders as a child

Value	Renders as
Strings and numbers	Text (so 0 shows as 0)
Elements	DOM
Arrays	Each item, in order
<code>null</code> , <code>undefined</code> , <code>true</code> , <code>false</code>	Nothing: this is what makes conditional rendering work
A plain object	An error: "Objects are not valid as a React child"; show one of its fields

```
function App() {
  const task = { title: 'Buy milk', due: '2026-10-07', done: false };
  const tags = ['home', 'shopping'];
  const daysLeft = 2;
  return (
    <article>
      <h2>{task.title.toUpperCase()}</h2>
      <p>Due {task.due}: {daysLeft * 24} hours left</p>
      <p>Status: {task.done ? 'done' : 'to do'}</p>
      <p>Tags: {tags.join(', ')}</p>
      <p>{null}{false}{undefined}(nothing above renders)</p>
    </article>
  );
}
```

Example. In the Try it box, add `<p>{task}</p>`: the preview shows the error "Objects are not valid as a React child (found: object with keys {title, due, done})". React cannot guess how to display an object; `{task.title}` or `{JSON.stringify(task)}` (for debugging) can be displayed.

Common mistake. Putting an `if` inside JSX: `{if (done) { <p>Done</p> }}` is a syntax error, because braces expect a value. Compute it before the return (`let badge = null; if (done) badge = <p>Done</p>; then {badge}`), or use an expression: `{done ? <p>Done</p> : null}`. See [Conditional rendering](#).

Props and composition

Props: passing data into a component

Props (properties) are the inputs of a component: the attributes you write on its tag arrive as one object, the first parameter of the function.

- **Any value:** strings in quotes; numbers, booleans, arrays, objects, elements and functions in braces. A function prop is how a child tells its parent that something happened (see [Events](#)).
- **A missing prop** is undefined: give a default in the destructuring when a sensible one exists, `{ size = 'medium' }`.
- **A bare attribute**, `<Badge urgent />`, passes `urgent: true`.
- **One object or separate fields:** `task={task}` is shorter; `title={...} done={...}` makes the dependencies explicit. Pick one style per component.

```
function TaskItem({ title, done, priority = 'normal' }) {
  return (
    <li>
      {done ? '✓' : 'o'} {title}
      {priority === 'high' && <strong> (urgent)</strong>}
    </li>
  );
}

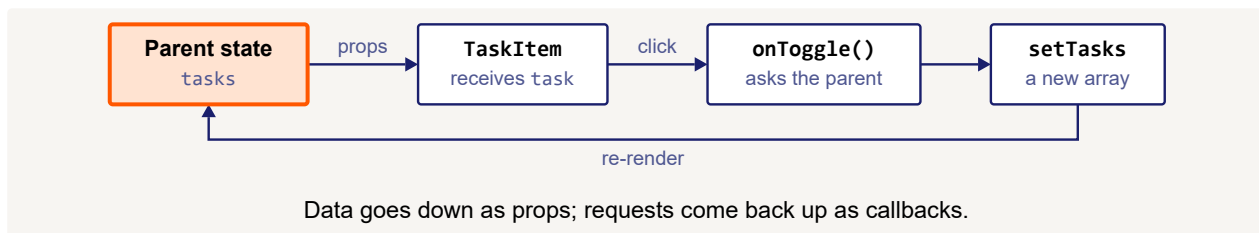
function App() {
  return (
    <ul>
      <TaskItem title="Buy milk" done={true} />
      <TaskItem title="Call Ana" done={false} priority="high" />
      <TaskItem title="Water the plants" />
    </ul>
  );
}
```

Example. The third `TaskItem` passes no `done`, so `done` is undefined (falsy) and the item shows `o`; it passes no `priority`, so the default `'normal'` applies. Change `done={true}` to `done="false"`: the first item now shows `✓`, because the **string** `"false"` is truthy. Quotes pass strings; braces pass values.

Common mistake. Writing `<TaskItem done="false" count="3" />` and expecting a boolean and a number. Quoted props are always strings: `"false"` is truthy and `"3" + 1` is `"31"`. Use braces for anything that is not text: `done={false}, count={3}`.

Props are read-only: data flows down

A component must never change its props: data flows **one way**, from parent to child, and a child that needs a change asks the parent by calling a function the parent passed down.



```
import { useState } from 'react';

function TaskItem({ task, onToggle }) {
  // task.done = !task.done; ← never: props are read-only
  return (
    <li>
      <label>
        <input type="checkbox" checked={task.done} onChange={onToggle} />
        {task.title}
      </label>
    </li>
  );
}

function App() {
  const [tasks, setTasks] = useState([
    { id: 't1', title: 'Buy milk', done: false },
    { id: 't2', title: 'Call Ana', done: true },
  ]);
  function toggle(id) {
    setTasks(tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  }
  const left = tasks.filter((t) => !t.done).length;
  return (
    <>
```

```

    <p>{left} left</p>
    <ul>
      {tasks.map((t) => <TaskItem key={t.id} task={t} onToggle={() => toggle(t.id)} />)}
    </ul>
  </>
);
}

```

Example. Tick "Buy milk" in the Try it box. `TaskItem` does not change anything itself: it calls `onToggle`, the parent's `toggle` builds a **new** array with a new task object, and React re-renders `App`, so both the checkbox and the "left" counter update together. The counter could never update if the child had flipped `task.done` on its own copy.

Common mistake. Copying a prop into the child's own state "so it can be edited": `const [title, setTitle] = useState(props.title)`. The copy is taken once, so when the parent later passes a new title the child keeps showing the old one, and now two components disagree about the data. Read the prop directly, and send changes up with a callback (see [State, props and derived values](#)).

children: components that wrap other content

Whatever you put **between** a component's opening and closing tags arrives as a special prop, `children`, so you can build wrappers (cards, panels, layouts, dialogs) that do not need to know what they contain.

- **Composition instead of options:** rather than one component with a prop for every variation (`showFooter`, `footerText`, `icon...`), build small pieces and nest them.
- **Layouts** are the classic case: a `Layout` renders the header and the navigation, and puts each page in between with `{children}`. Forms, dialogs and list sections follow the same pattern.
- **More than one slot:** pass elements through ordinary props as well, `<Panel title="Tasks" actions={<button>Add</button>}>...</Panel>`.

```

function Card({ title, children }) {
  return (
    <section className="card">
      <h2>{title}</h2>
      {children}
    </section>
  );
}

function App() {
  return (
    <>
      <Card title="Today">
        <p>Buy milk</p>
        <p>Call Ana</p>
      </Card>
      <Card title="Notes">
        <em>Nothing yet.</em>
      </Card>
    </>
  );
}

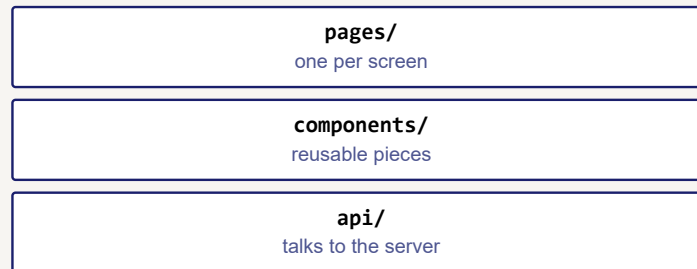
```

Example. Both cards share the border, padding and heading style, but contain different content: two paragraphs in the first, an `<em>` in the second. Remove `{children}` from `Card`: the titles stay but the content disappears, because nothing tells React where to put it.

Common mistake. Forgetting to destructure or render `children`: the component is written as `function Card({ title })`, the content between the tags is silently dropped, and there is no error. If content you wrote between tags is missing, check that the wrapper renders `{children}`.

Composing components and splitting files

Build screens by **composing** small components, and give each component its own file, connected with **export** and **import**.



Each folder has one job; screens are built from the pieces below them.

pages/ One component per screen: the task list, the task detail, the log-in page.

components/ Reusable pieces that pages are built from.

api/, hooks/ Code that is not UI: requests to the server, reusable logic.

```

// src/components/TaskItem.jsx
export default function TaskItem({ task }) {
  return <li>{task.title}</li>;
}

// src/components/TaskList.jsx
import TaskItem from './TaskItem.jsx';

export default function TaskList({ tasks }) {
  return (
    <ul>
      {tasks.map((task) => <TaskItem key={task.id} task={task} />)}
    </ul>
  );
}

// src/pages/TasksPage.jsx
import TaskList from '../components/TaskList.jsx';

export default function TasksPage() {
  const tasks = [{ id: 't1', title: 'Buy milk' }];
  return <TaskList tasks={tasks} />;
}

/* src/
  main.jsx      renders <App /> into #root
  App.jsx
  pages/        one component per screen
  components/   reusable pieces
  api/          code that talks to the server    */
  
```

Example. The tasks page imports `TaskList`, which imports `TaskItem`: the import graph mirrors the component tree. If a teammate needs the same task row on a "Search results" page, they import `TaskItem` there too, and a fix to it shows up in both places.

Common mistake. Mixing up default and named imports: `import { TaskItem } from './TaskItem.jsx'` when the file has `export default` gives undefined, and React then fails with "Element type is invalid: expected a string ... but got: undefined". When you see that error, check the import against the export, and the file path.

Lists and conditions

Rendering lists with map

To show a list, turn the array of data into an array of elements with `array.map(item => <Element ... />)`; React renders every element of an array, in order.

- `map`, **not** `forEach`: `map` returns the new array; `forEach` returns undefined, which renders nothing.
- **Prepare, then map**: `filter` to keep some items, `toSorted` to order them. Never sort the original array: it may be a prop or state. Compute the result in a variable above the return so the JSX stays readable.
- A key **on each item** tells React which item is which between renders (see [Keys](#)). It goes on the element in the map, `<TaskItem key={task.id} task={task} />`, not inside `TaskItem`.

```
const tasks = [
  { id: 't1', title: 'Buy milk', done: true },
  { id: 't2', title: 'Call Ana', done: false },
  { id: 't3', title: 'Water the plants', done: false },
];

function App() {
  const pending = tasks.filter((t) => !t.done);
  return (
    <>
      <h2>To do ({pending.length})</h2>
      <ul>
        {pending.map((task) => (
          <li key={task.id}>{task.title}</li>
        ))}
      </ul>
    </>
  );
}
```

Example. The box filters out done tasks, then maps the rest: two `<li>`. Change `map` to `forEach`: the list is empty and there is no error, because `forEach` returns undefined, which renders nothing.

Common mistake. Using curly braces in the arrow function and forgetting `return`: `tasks.map((t) => { <li>{t.title}</li> })` produces an array of undefined, so nothing renders. With braces the body is a block and needs `return`; with parentheses, `(t) => (<li>...</li>)`, the element is returned implicitly.

Keys: telling list items apart

A **key** is a string or number, unique among siblings, that identifies a list item across renders, so React keeps each item's DOM and state with the right item.

Choosing a key

Key	The same item keeps it?	Verdict
<code>task.id</code> , from the data	Yes	The right choice
<code>crypto.randomUUID()</code> , when the item is created	Yes: stored with the item	Right for items made in the browser
<code>index</code> , the position	No: it changes on insert, remove or reorder	Only for lists that never change order or length
<code>Math.random()</code> in the render	No: new on every render	Never: every row is recreated each time

- **Unique among siblings**, not across the whole app: two lists can reuse the same ids.
- **No key at all**: React falls back to the index and warns "Each child in a list should have a unique key prop".
- **For React only**: the key is not passed to the component as a prop. Changing a component's key on purpose resets it (see [State belongs to a place in the tree](#)).

```
import { useState } from 'react';

function Row({ title }) {
  return (
    <li>
      {title} <input placeholder="note" />
    </li>
  );
}

function App() {
  const [tasks, setTasks] = useState([
    { id: 'a', title: 'Buy milk' },
    { id: 'b', title: 'Call Ana' },
  ]);
  function addFirst() {
    const id = crypto.randomUUID();
    setTasks([
      { id, title: 'New task ' + id.slice(0, 4) },
      ...tasks
    ]);
  }
  return (
    <>
      <button onClick={addFirst}>Add at the top</button>
      <ul>
        {tasks.map((task, index) => (
          <Row key={index} title={task.title} />
        ))}
      </ul>
    </>
  );
}
```

Example. In the Try it box, type "2 litres" in the note next to "Buy milk", then press **Add at the top**. The note stays in the **first** row, now next to the new task: with `key={index}` React thinks row 0 is the same item as before and keeps its input. Change it to `key={task.id}` (the preview starts afresh), repeat, and the note travels with "Buy milk".

Common mistake. Silencing the warning with `key={index}` or `key={Math.random()}`. The first hides the bug until the list is reordered, filtered or prepended (wrong rows keep the wrong inputs, checkboxes or animations); the second destroys every row on every render (inputs lose focus while you type). Use an id that belongs to the item.

Conditional rendering: if, ternary and &&

To show different UI for different data, use plain JavaScript: an **early return** for whole alternatives, a **ternary** for either/or inside JSX, and **&&** for "this or nothing".

Four tools, from the whole component down to one element

Tool	Use it for	Example
Early return	A whole alternative: loading, error, empty	<code>if (tasks.length === 0) return <p>No tasks yet.</p>;</code>
Ternary ? :	Either this or that, inside JSX	<code>{done ? <s>{title}</s> : title}</code>
&&	This or nothing	<code>{isAdmin && <button>Delete</button>}</code>
return null	A component hiding itself	<code>if (!badge) return null;</code>

```
function TaskList({ tasks, isAdmin }) {
  if (tasks.length === 0) {
    return <p>No tasks yet.</p>;
  }
  return (
    <ul>
      {tasks.map((t) => (
        <li key={t.id}>
          {t.done ? <s>{t.title}</s> : t.title}
          {isAdmin && <button>Delete</button>}
        </li>
      ))}
    </ul>
  );
}
```

```

    );
  }

function App() {
  const tasks = [{ id: 't1', title: 'Buy milk', done: true }];
  const none = [];
  return (
    <>
      <TaskList tasks={tasks} isAdmin={true} />
      <TaskList tasks={none} />
      <p>Trap: [{none.length && <b>never shown</b>}]</p>
    </>
  );
}

```

Example. The last line of the Try it box shows `[0]: none.length` is `0`, so `0 && ...` evaluates to `0` and React prints it. Change it to `none.length > 0 && ...` and the brackets are empty. The empty `TaskList` uses an early return, so its main JSX never has to think about the empty case.

Common mistake. Writing `{count && <Badge count={count} />}` for a counter that can be zero, or `{items.length && ...}` for a list. The page shows a lone `0` exactly when there is nothing to show. Compare explicitly `(count > 0)` or use a ternary.

Events and forms

Events: pass a function, do not call it

Event props such as `onClick`, `onChange` and `onSubmit` take a function that React calls when the event happens: `onClick={save}`, **not** `onClick={save()}`, **which calls it during render.**

`onClick={() => remove(task.id)}` Passes an argument: the arrow creates a function that calls `remove` later, with the right id.

e, the event object

React's wrapper around the browser event, with the same main fields: `e.target`, `e.target.value`, `e.preventDefault()`, `e.stopPropagation()` (see [The event object](#) and [stopPropagation](#)).

camelCase names

`onClick`, `onChange`, `onKeyDown`, `onMouseEnter`, on HTML elements.

onX on your own components

Just a prop name you choose: `<TaskItem onDelete={...} />` does nothing until `TaskItem` passes it to a real element or calls it. Name handlers `handleX` inside a component and props `onX`.

```

import { useState } from 'react';

function App() {
  const [log, setLog] = useState([]);
  function add(text) {
    setLog([...log, text]);
  }
  function handleClick(e) {
    add('clicked ' + e.target.textContent);
  }
  return (
    <>
      <button onClick={handleClick}>Save</button>{' '}
      <button onClick={() => add('deleted task t1')}>Delete t1</button>{' '}
      <button onClick={() => setLog([])}>Clear</button>
      <ul>
        {log.map((line, i) => <li key={i}>{line}</li>)}
      </ul>
    </>
  );
}

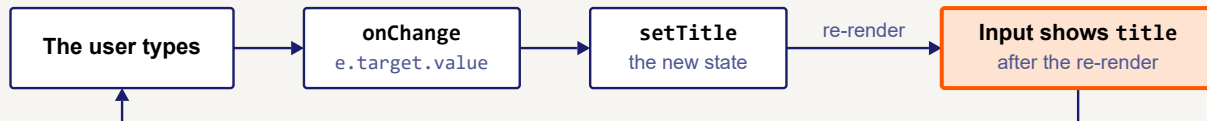
```

Example. Each button in the Try it box adds a line. `handleClick` receives the event and reads the button text; the Delete button needs an argument, so it is wrapped in an arrow. Change `onClick={() => setLog([])}` to `onClick={setLog([])}`: the preview crashes with "Too many re-renders", because clearing during render triggers another render, forever. (The log uses the index as key: acceptable here, lines are only appended, never reordered.)

Common mistake. Writing `onClick={handleDelete(task.id)}` to pass the id. It calls `handleDelete` for every task as soon as the list renders, deleting everything or looping. Use `onClick={() => handleDelete(task.id)}`.

Controlled inputs: value and onChange

A controlled input shows a value from state (`value={title}`) and reports every keystroke back (`onChange={(e) => setTitle(e.target.value)}`), so the state is the single source of truth for what the user typed.



The input only displays the state; every keystroke goes round the loop.

- **Everything else becomes easy:** a live preview, a character count, disabled=`{title.trim() === ''}`, clearing after saving with `setTitle('')`, forcing uppercase with `setCode(e.target.value.toUpperCase())`.
- **Checkboxes** use `checked` and `e.target.checked`; `<select>` and `<textarea>` use `value` like an input.

Two warnings that mean the loop is broken

value without onChange

The input is read-only: every keystroke is undone, and React warns "You provided a value prop to a form field without an onChange handler".

A value that starts as undefined

React warns "A component is changing an uncontrolled input to be controlled". Start the state as `''`, not `undefined`.

```
import { useState } from 'react';

function App() {
  const [title, setTitle] = useState('');
  const [urgent, setUrgent] = useState(false);
  const empty = title.trim() === '';
  return (
    <div>
      <label htmlFor="t">Title </label>
      <input id="t" value={title} onChange={(e) => setTitle(e.target.value)} />
      <label>
        <input type="checkbox" checked={urgent} onChange={(e) => setUrgent(e.target.checked)} />
        Urgent
      </label>
      <p>Preview: {empty ? '(empty)' : title}{urgent && ' (urgent)'} · {title.length}/40</p>
      <button disabled={empty} onClick={() => setTitle('')}>Clear</button>
    </div>
  );
}
```

Example. Type in the Try it box: the preview and the counter follow every keystroke, and **Clear** empties the input because the input displays `title`. Remove the `onChange` from the text input: typing does nothing (the input keeps showing `title`, which never changes) and the console shows React's warning about value without `onChange`.

Common mistake. Writing `value={title}` and reading the text later with `document.getElementById('t').value`, or forgetting `onChange`. In React the state is the truth: if it does not change, the input cannot change. Either control the input fully (`value + onChange`) or leave it uncontrolled and read it on submit (see [Uncontrolled inputs](#)).

Uncontrolled inputs: defaultValue and FormData

An **uncontrolled input** keeps its own value in the DOM: you give it a starting value with `defaultValue` and read every field once, on submit, with `new FormData(e.target)`.

`defaultValue="..."`, `defaultChecked` The starting value. React sets it once and then leaves the input alone.

`new FormData(e.target)` Reads every named field of the submitted form.

`Object.fromEntries(formData)` Turns it into a plain object such as `{ email: '...', password: '...' }`. A checked box gives `'on'`; an unchecked one is left out.

`e.target.reset()` Puts every field back to its default value.

Controlled or uncontrolled?

	Controlled	Uncontrolled
The value lives in	State	The DOM
You read it	On every keystroke, through <code>value</code>	Once, on submit, through <code>FormData</code>
Good for	Live previews, checks as you type, fields that depend on each other	Simple forms read once: log-in, search
Re-renders while typing	Yes, every keystroke	No

```
import { useState } from 'react';

function App() {
  const [sent, setSent] = useState(null);

  function handleSubmit(e) {
    e.preventDefault();
    const data = Object.fromEntries(new FormData(e.target));
    setSent(data);
    // back to the default values
    e.target.reset();
  }

  return (
    <form onSubmit={handleSubmit}>
      <p><label>Email <input name="email" type="email" defaultValue="ana@example.com" /></label></p>
      <p><label>Password <input name="password" type="password" /></label></p>
      <p><label><input name="remember" type="checkbox" defaultChecked /> Remember me</label></p>
      <button>Log in</button>
      {sent && <pre>{JSON.stringify(sent, null, 2)}</pre>}
    </form>
  );
}
```

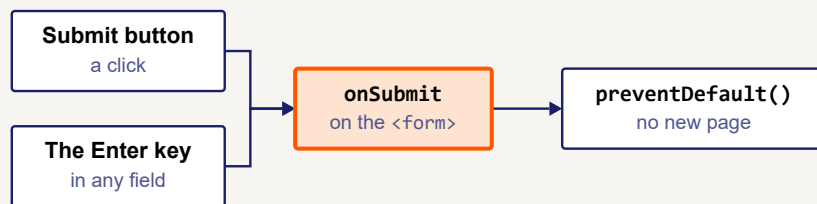
Example. In the Try it box, type a password and press **Log in**: the object shows all three fields, `remember: "on"`, and the form goes back to its defaults. Untick **Remember me** and log in again: `remember` is missing from the object, because `FormData` leaves unchecked boxes out. Nothing re-rendered while you typed.

Common mistake. Writing `value="ana@example.com"` instead of `defaultValue`. `value` makes the input controlled: without an `onChange` it is read-only, and React warns about it. Use `defaultValue` for a starting value you will read on submit, or control the input fully (see [Controlled inputs](#)).

Forms: onSubmit and preventDefault

Handle a form with `onSubmit` on the `<form>`, not `onClick` on the button, and call `e.preventDefault()` first: by default

the browser submits the form by loading a new page, which wipes your app's state.



Both ways of submitting reach onSubmit;
onClick on the button sees only one of them.

1. **Prevent** the default: `e.preventDefault()`.
2. **Validate**: trim the text; if it is empty or invalid, show a message and return early.
3. **Use** the data: add it to the list, or send it to an API.
4. **Reset** the fields.

```

import { useState } from 'react';

function App() {
  const [title, setTitle] = useState('');
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    const clean = title.trim();
    if (!clean) {
      setError('Write a title first. ');
      return;
    }
    setTasks([...tasks, { id: crypto.randomUUID(), title: clean }]);
    setTitle('');
    setError('');
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="title">New task </label>
      <input id="title" value={title} onChange={(e) => setTitle(e.target.value)} />
      <button type="submit">Add</button>
      <button type="button" onClick={() => setTasks([])}>Clear all</button>
      {error && <p role="alert">{error}</p>}
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </form>
  );
}

```

Example. Add two tasks in the Try it box, once with the button and once by pressing Enter: both go through onSubmit. Submit an empty title: the error appears and nothing is added. Now delete the line `e.preventDefault()`; and add a task: the browser submits the form, the preview reloads and both the list and the input are gone.

Common mistake. Putting the logic in `onClick` of the submit button and skipping `preventDefault`. Pressing Enter in the input bypasses the click handler, and the default submission reloads the page; the symptom is "my list flashes and disappears". Use `onSubmit` on the form with `e.preventDefault()` as its first line.

Forms with several fields and field errors

For a form with several inputs, keep **one state object** with a field per input and one change handler that uses each input's name; keep the errors in a second object with the **same keys**.

values

`{ title: '', due: '' }`: one key per field. Clearing the form is `setValues(EMPTY)`; a form that creates and edits only changes the initial value.

errors

The same keys, holding messages: { title: 'Title is required' }. Each message renders under its own field.

aria-invalid, aria-describedby

Connect each field to its message for screen readers (see [ARIA attributes and keyboard focus](#)).

```
import { useState } from 'react';

const EMPTY = { title: '', due: '' };

function validate(v) {
  const errors = {};
  if (!v.title.trim()) errors.title = 'Title is required';
  if (v.due && v.due < '2026-01-01') errors.due = 'Pick a date from 2026';
  return errors;
}

function App() {
  const [values, setValues] = useState(EMPTY);
  const [errors, setErrors] = useState({});
  const [saved, setSaved] = useState(null);
  function handleChange(e) {
    setValues({ ...values, [e.target.name]: e.target.value });
  }
  function handleSubmit(e) {
    e.preventDefault();
    const found = validate(values);
    setErrors(found);
    if (Object.keys(found).length > 0) return;
    setSaved(values);
    setValues(EMPTY);
  }
  return (
    <form onSubmit={handleSubmit} noValidate>
      <p><label>Title <input name="title" value={values.title} onChange={handleChange}
        aria-invalid={!!errors.title} aria-describedby="e-title" /></label>
        <small id="e-title"> {errors.title}</small></p>
      <p><label>Due <input type="date" name="due" value={values.due} onChange={handleChange}
        aria-invalid={!!errors.due} aria-describedby="e-due" /></label>
        <small id="e-due"> {errors.due}</small></p>
      <button>Save</button>
      {saved && <p>Saved: {JSON.stringify(saved)}</p>}
    </form>
  );
}
```

Example. Press **Save** with an empty form: only the title error appears, under the title. Pick a date in 2025 and save again: now the date error shows as well. Fill both correctly: the object is saved and the form resets in one call. If a server answered 400 with errors: [{ field: 'title', message: 'Title already used' }], turning that list into { title: 'Title already used' } would show the message in the same place.

Common mistake. Forgetting the spread: setValues({ [e.target.name]: e.target.value }) replaces the whole object with one field, so typing in "Due" erases the title (and the title input switches to uncontrolled, with a warning). Always copy the rest: { ...values, [name]: value }.